CSC 1202 — ILES Exam Study Sheet (2026)

Goal: memorise enough syntax to write Django + React code with a pen, no IDE, no autocomplete. The exam reuses AITS patterns — only the case study is new (ILES instead of AITS). Every block below is something that has appeared in or is one tiny step from a 2025 exam question.

---

0. Pen-and-paper survival rules

1. **Write line numbers in the margin** if the question says "fill line 3, 7, 12" — don't get them out of order.
2. **For fill-in questions, write the WHOLE line, not just the missing token.** Markers grade what's on the page.
3. **Indent Python with 4 spaces consistently.** Mixed tabs/spaces = wrong on paper too — they can see it.
4. **Close every bracket, brace, parenthesis, JSX tag.** Run a finger over each opener and find its closer before moving on.
5. **In React, `useState` initialises state, `useEffect` runs side effects.** AITS swapped them as a trap in two separate questions — don't fall for it.
6. **Always import what you use.** Even on paper. `from rest_framework.views import APIView`, `from django.db import models`, `from django.contrib.auth.models import AbstractUser`.
7. **Add a one-line comment when asked** (Q5 last year demanded it). Format: `# This filters issues by the logged-in user`.
8. **Status codes:** 200 OK, 201 Created, 204 No Content, 400 Bad, 401 Unauth, 403 Forbidden, 404 Not Found.
9. **HTTP verbs:** GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove).
10. **Five questions, three hours = ~35 min each.** Don't camp on one. Move on if stuck.

---

1. ILES domain you must know cold

Four roles: **Student Intern**, **Workplace Supervisor**, **Academic Supervisor**, **Internship Administrator**.

Core entities (use these names in answers unless the question gives different ones):

- `CustomUser` (extends `AbstractUser`) — `role`, `department`
- `Department`
- `InternshipPlacement` — student, company, supervisor, start_date, end_date
- `WeeklyLog` — student, placement, week_number, activities, hours, status
- `EvaluationCriteria` — name, weight (e.g. 0.4 for 40%)
- `Evaluation` — student, criteria, score, evaluator

Workflow states for `WeeklyLog`: **Draft -> Submitted -> Reviewed -> Approved** (rejection sends it back to Draft).

Weighted scoring: total = (workplace * 0.4) + (academic * 0.3) + (logbook * 0.3) — example only, the question will give you the exact weights.

---

2. Django models — the syntax block you must own

```python
from django.db import models
from django.contrib.auth.models import AbstractUser

class Department(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name


class CustomUser(AbstractUser):
    ROLE_CHOICES = [
        ('student', 'Student'),
        ('workplace_supervisor', 'Workplace Supervisor'),
        ('academic_supervisor', 'Academic Supervisor'),
        ('admin', 'Administrator'),
    ]
    role = models.CharField(max_length=30, choices=ROLE_CHOICES)
    department = models.ForeignKey(
        Department, on_delete=models.SET_NULL,
        null=True, blank=True, related_name='users'
    )

    def __str__(self):
        return f"{self.username} ({self.role})"


class InternshipPlacement(models.Model):
    student = models.ForeignKey(
        CustomUser, on_delete=models.CASCADE, related_name='placements'
    )
    company = models.CharField(max_length=150)
    supervisor = models.ForeignKey(
        CustomUser, on_delete=models.SET_NULL, null=True,
        related_name='supervised_placements'
    )
    start_date = models.DateField()
    end_date = models.DateField()


class WeeklyLog(models.Model):
    STATUS_CHOICES = [
        ('Draft', 'Draft'),
        ('Submitted', 'Submitted'),
        ('Reviewed', 'Reviewed'),
        ('Approved', 'Approved'),
    ]
    placement = models.ForeignKey(
        InternshipPlacement, on_delete=models.CASCADE, related_name='logs'
    )
    week_number = models.PositiveIntegerField()
    activities = models.TextField()
    hours = models.PositiveIntegerField(default=0)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='Draft')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ('placement', 'week_number')
        ordering = ['-week_number']

    def __str__(self):
        return f"Week {self.week_number} — {self.placement.student.username}"
```

Field types — memorise these names

Field	Use
`CharField(max_length=N)`	Short text. Always need max_length.
`TextField()`	Long text. No max_length.
`IntegerField()` / `PositiveIntegerField()`	Integers.
`FloatField()`	Float.
`DecimalField(max_digits=5, decimal_places=2)`	Money / scores.
`BooleanField(default=False)`	Yes/no.
`DateField()` / `DateTimeField()`	Dates. `auto_now_add=True` on create, `auto_now=True` on every save.
`EmailField()`	Validated email.
`URLField()`	Validated URL.
`FileField(upload_to='path/')`	File upload.
`ImageField()`	Image upload.

Relationships — the ones that appear on exams

```
# ONE-to-MANY  — many logs belong to one placement
placement = models.ForeignKey(Placement, on_delete=models.CASCADE,
                              related_name='logs')

# ONE-to-ONE  — one profile per user
user = models.OneToOneField(User, on_delete=models.CASCADE)

# MANY-to-MANY — a student can have many criteria
criteria = models.ManyToManyField(EvaluationCriteria, related_name='students')
```

`on_delete` options (pick the right one)

- `CASCADE` — delete the child when the parent is deleted (most common).
- `SET_NULL` — set FK to NULL (needs `null=True`). Use when child should survive.
- `PROTECT` — block deletion of parent.
- `SET_DEFAULT` — use `default=` value.
- `DO_NOTHING` — don't touch (rare).

`related_name`

Lets you reverse-traverse. `student.placements.all()` works because `related_name='placements'` was set on the FK from Placement to student.

`__str__`

Always implement it. Returns a string. Markers love it. One line. `return self.name` or `return f"..."`.

---

3. Django ORM — query patterns

```
Model.objects.all()
Model.objects.filter(status='Approved')
Model.objects.exclude(status='Resolved')
Model.objects.get(pk=1)            # raises if 0 or >1
Model.objects.first()
Model.objects.count()
Model.objects.order_by('-created_at')

# Field lookups — append __<lookup>
WeeklyLog.objects.filter(week_number__gte=5)
WeeklyLog.objects.filter(activities__icontains='design')
WeeklyLog.objects.filter(status__in=['Submitted', 'Reviewed'])

# Traverse relationships with __
WeeklyLog.objects.filter(placement__student__department=request.user.department)

# Combine excludes / filters
WeeklyLog.objects.filter(
    placement__student__department=request.user.department
).exclude(status='Approved')

# OR with Q
from django.db.models import Q
WeeklyLog.objects.filter(Q(status='Submitted') | Q(status='Reviewed'))

# Aggregate
from django.db.models import Count, Avg, Sum
WeeklyLog.objects.aggregate(total=Count('id'), avg_hours=Avg('hours'))
# Annotate per row
Placement.objects.annotate(num_logs=Count('logs'))
```

The Q1c-style query

> "All unresolved logs submitted by students in the same department as the registrar."

```
WeeklyLog.objects.filter(
    placement__student__department=request.user.department
).exclude(status='Approved')
```

4. DRF — serializers

```python
from rest_framework import serializers
from .models import WeeklyLog

class WeeklyLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = WeeklyLog
        fields = ['id', 'placement', 'week_number', 'activities',
                  'hours', 'status']
        # OR
        # fields = '__all__'
        read_only_fields = ['status']

    # Single-field validation
    def validate_week_number(self, value):
        if value < 1 or value > 52:
            raise serializers.ValidationError("Week must be 1–52.")
        return value

    # Cross-field validation
    def validate(self, data):
        if not data.get('activities'):
            raise serializers.ValidationError(
                "Activities cannot be empty."
            )
        if data.get('hours', 0) > 80:
            raise serializers.ValidationError(
                "Hours per week cannot exceed 80."
            )
        return data
```

Key rules:

- `class Meta` must declare `model` and `fields`.
- `fields` is a list (or `'__all__'`). It is NOT a tuple in 2026 syntax — list is safer.
- Single-field validator is `validate_<fieldname>(self, value)` — return `value`.
- Cross-field validator is `validate(self, data)` — return `data`.
- Raise `serializers.ValidationError("message")` — not Python's `ValueError`.

5. DRF — views

APIView (the one AITS asked you to write)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticated
from .models import WeeklyLog
from .serializers import WeeklyLogSerializer

class StudentLogsView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        logs = WeeklyLog.objects.filter(
            placement__student=request.user
        )
        serializer = WeeklyLogSerializer(logs, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = WeeklyLogSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save(
                placement=request.user.placements.first()
            )
            return Response(serializer.data,
                            status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
                        status=status.HTTP_400_BAD_REQUEST)
```

Memorise the four lines that appear in nearly every view:

```
serializer = MySerializer(data=request.data)
if serializer.is_valid():
    serializer.save()
    return Response(serializer.data, status=status.HTTP_201_CREATED)
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Generic views (shorter alternative)

```
from rest_framework import generics

class LogListCreate(generics.ListCreateAPIView):
    serializer_class = WeeklyLogSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return WeeklyLog.objects.filter(
            placement__student=self.request.user
        )
```

URL wiring

```
# project/urls.py
from django.urls import path, include
urlpatterns = [
    path('api/', include('logs.urls')),
]

# logs/urls.py
from django.urls import path
from .views import StudentLogsView

urlpatterns = [
    path('student-logs/', StudentLogsView.as_view(), name='student-logs'),
]
```

---

6. Authentication & RBAC

```
from rest_framework.permissions import BasePermission

class IsStudent(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and \
               request.user.role == 'student'

class IsOwnerOrSupervisor(BasePermission):
    def has_object_permission(self, request, view, obj):
        return obj.placement.student == request.user or \
               obj.placement.supervisor == request.user

# Use it:
class MyView(APIView):
    permission_classes = [IsAuthenticated, IsStudent]
```

In settings:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
AUTH_USER_MODEL = 'accounts.CustomUser'
```

---

7. Django signals — `post_save` pattern AITS asked

```python
from django.db.models.signals import post_save, pre_save
from django.dispatch import receiver
from django.core.mail import send_mail
from .models import WeeklyLog

@receiver(post_save, sender=WeeklyLog)
def notify_on_approval(sender, instance, created, **kwargs):
    # `created` is True on first save, False on update.
    # We only want UPDATES that flip status to 'Approved'.
    if not created and instance.status == 'Approved':
        send_mail(
            subject="Log Approved",
            message=f"Your week {instance.week_number} log was approved.",
            from_email="admin@iles.com",
            recipient_list=[instance.placement.student.email],
        )
```

Bugs to watch for (these appeared in AITS Q4):

- Missing `created` parameter → `created` undefined.
- Forgetting `if not created:` → email fires on first save too.
- Comparing status without ensuring it's a string (`==` not `=`).
- `recipient_list` must be a **list**, not a string.
- Decorator typo: `@receiver(post_save, sender=WeeklyLog)` — `sender=` is mandatory.

To detect status change (only fire when it transitions to Approved), use `pre_save` to capture old value:

```python
@receiver(pre_save, sender=WeeklyLog)
def stash_old_status(sender, instance, **kwargs):
    if instance.pk:
        instance._old_status = WeeklyLog.objects.get(pk=instance.pk).status
    else:
        instance._old_status = None

@receiver(post_save, sender=WeeklyLog)
def notify_on_status_change(sender, instance, created, **kwargs):
    if not created and instance._old_status != 'Approved' \
       and instance.status == 'Approved':
        send_mail(...)
```

Wire signals in `apps.py`:

```python
class LogsConfig(AppConfig):
    name = 'logs'
    def ready(self):
        from . import signals  # noqa
```

---

8. Django tests — exact pattern they grade

```python
from django.test import TestCase
from django.urls import reverse
from django.contrib.auth import get_user_model
from rest_framework.test import APITestCase, APIClient
from rest_framework import status

User = get_user_model()

class WeeklyLogAPITest(APITestCase):

    def setUp(self):
        self.student = User.objects.create_user(
            username='alice', password='pass123', role='student'
        )
        self.client = APIClient()
        self.client.login(username='alice', password='pass123')

    def test_student_can_create_log(self):
        data = {
            'week_number': 1,
            'activities': 'Read codebase',
            'hours': 20,
        }
        response = self.client.post('/api/logs/', data, format='json')
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)

    def test_unauthenticated_user_blocked(self):
        self.client.logout()
        response = self.client.get('/api/logs/')
        self.assertEqual(response.status_code, 401)

    def test_supervisor_cannot_create_log(self):
        supervisor = User.objects.create_user(
            username='bob', password='pass123', role='workplace_supervisor'
        )
        self.client.force_login(supervisor)
        response = self.client.post('/api/logs/', {}, format='json')
        self.assertIn(response.status_code, [403, 400])
```

Bugs AITS asked you to fix in tests:

- Used `User.objects.create()` instead of `create_user()` → password not hashed, `login()` fails.
- Forgot `self.client.login(...)` before the action being tested.
- Wrong expected status code (`200` instead of `403`).
- Test method name doesn't start with `test_` → not discovered.

9. Django settings + Heroku deployment

```
# settings.py — PRODUCTION
import os
import dj_database_url

SECRET_KEY = os.environ.get('SECRET_KEY')
DEBUG = os.environ.get('DEBUG', 'False') == 'True'
ALLOWED_HOSTS = os.environ.get('ALLOWED_HOSTS', '').split(',')

DATABASES = {
    'default': dj_database_url.config(
        default=os.environ.get('DATABASE_URL'),
        conn_max_age=600,
    )
}

STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # ...
]
```

Procfile (no extension, in project root)

```
web: gunicorn projectname.wsgi --log-file -
release: python manage.py migrate --noinput
```

runtime.txt

```
python-3.11.6
```

.env (LOCAL ONLY — never committed)

```
SECRET_KEY=averylongrandomstring
DEBUG=False
ALLOWED_HOSTS=yourapp.herokuapp.com,localhost
DATABASE_URL=postgres://user:pass@host:5432/dbname
```

Bugs AITS asked you to fix in settings

- `DEBUG = True` in production → leaks stack traces. Fix: `DEBUG = False`.
- `ALLOWED_HOSTS = []` → all requests rejected when DEBUG is False. Fix: `['yourapp.herokuapp.com']`.
- `SECRET_KEY = 'hard-coded-string'` → leak risk. Fix: `os.environ.get('SECRET_KEY')`.
- `STATIC_ROOT = ''` → `collectstatic` fails. Fix: `os.path.join(BASE_DIR, 'staticfiles')`.
- Procfile pointing at the wrong wsgi module, missing `gunicorn`, or missing `--log-file -`.
- `.env` shipped with `DEBUG=True` and `ALLOWED_HOSTS=*` — both insecure for prod.

10. React — the absolute essentials

Functional component

```
function StudentDashboard() {
  return (
    <div>
      <h1>My Logs</h1>
    </div>
  );
}
export default StudentDashboard;
```

`useState` (initialises and updates state)

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);     // 0 is the initial value
  const [logs, setLogs]   = useState([]);    // empty array for lists
  const [user, setUser]   = useState(null);  // null for "no value yet"
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

`useEffect` (run a side effect)

```
import { useEffect, useState } from 'react';

function LogList() {
  const [logs, setLogs] = useState([]);   // <-- useState, NOT useEffect

  useEffect(() => {
    fetch('/api/student-logs/')
      .then(res => res.json())
      .then(data => setLogs(data));
  }, []);  // empty array = run once on mount

  return (
    <ul>
      {logs.map(log => (
        <li key={log.id}>{log.week_number} — {log.status}</li>
      ))}
    </ul>
  );
}
```

The classic AITS trap (appeared twice):

```
const [issues, setIssues] = useEffect([]);   // WRONG
const [issues, setIssues] = useState([]);    // CORRECT
```

POST with `fetch`

```
const handleSubmit = async (e) => {
  e.preventDefault();
  const res = await fetch('/api/logs/', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(formData),
  });
  if (res.ok) {
    setSuccess(true);
  } else {
    const errBody = await res.json();
    setError(errBody.detail || 'Submission failed');
  }
};
```

Controlled form

```
function LogForm() {
  const [formData, setFormData] = useState({
    week_number: '', activities: '', hours: ''
  });
  const [success, setSuccess] = useState(false);
  const [error, setError]     = useState(null);
  const [submitting, setSubmitting] = useState(false);

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSubmitting(true);
    setError(null);
    try {
      const res = await fetch('/api/logs/', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });
      if (res.ok) {
        setSuccess(true);
      } else {
        const data = await res.json();
        setError(data.detail || 'Failed to submit.');
      }
    } catch (err) {
      setError(err.message);
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="week_number" value={formData.week_number}
             onChange={handleChange} />
      <textarea name="activities" value={formData.activities}
                onChange={handleChange} />
      <input name="hours" value={formData.hours}
             onChange={handleChange} />
      <button type="submit" disabled={submitting}>
        {submitting ? 'Submitting…' : 'Submit'}
      </button>
      {success && <p>Log submitted successfully!</p>}
      {error && <p className="error">{error}</p>}
    </form>
  );
}
```

Conditional rendering

```
{ success && <p>Done!</p> }                  // render if truthy
{ error ? <p>{error}</p> : <p>OK</p> }       // ternary
{ items.length === 0 && <p>No items</p> }
```

List rendering — never forget `key`

```
{logs.map(log => <li key={log.id}>{log.week_number}</li>)}
```

Toast on PATCH response (AITS Q4b style)

```
const updateLog = async (id) => {
  const res = await fetch(`/api/logs/${id}/`, { method: 'PATCH' });
  const data = await res.json();                // need to read body
  if (data.status === 'Approved') {             // === not =, .status not res.status
    Toast('Approved!');
  }
};
```

Bugs to watch for:

- `data.status = 'Approved'` (assignment) instead of `===` (comparison).
- `res.status` (HTTP code) instead of `data.status` (model field).
- Missing `await res.json()` → comparing a Promise to a string.

---

11. Weighted scoring (ILES-specific)

```
from decimal import Decimal

class Evaluation(models.Model):
    student = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
    workplace_score = models.DecimalField(max_digits=5, decimal_places=2)
    academic_score  = models.DecimalField(max_digits=5, decimal_places=2)
    logbook_score   = models.DecimalField(max_digits=5, decimal_places=2)
    total           = models.DecimalField(max_digits=5, decimal_places=2,
                                          blank=True, null=True)

    class Meta:
        unique_together = ('student',)   # one evaluation per student

    def save(self, *args, **kwargs):
        self.total = (
            self.workplace_score * Decimal('0.4') +
            self.academic_score  * Decimal('0.3') +
            self.logbook_score   * Decimal('0.3')
        )
        super().save(*args, **kwargs)
```

Decimals matter — multiplying a `DecimalField` by a Python `float` raises `TypeError`. Use `Decimal('0.4')`.

Preventing duplicate evaluation:

- `unique_together = ('student',)` at the DB level, OR
- In a serializer's `validate`: `python if Evaluation.objects.filter(student=data['student']).exists(): raise serializers.ValidationError("Already evaluated.")`

---

12. Workflow state transitions

```
ALLOWED_TRANSITIONS = {
    'Draft':     ['Submitted'],
    'Submitted': ['Reviewed', 'Draft'],
    'Reviewed':  ['Approved', 'Submitted'],
    'Approved':  [],
}

class WeeklyLog(models.Model):
    # ... fields ...

    def save(self, *args, **kwargs):
        if self.pk:
            old = WeeklyLog.objects.get(pk=self.pk)
            if old.status != self.status:
                if self.status not in ALLOWED_TRANSITIONS[old.status]:
                    raise ValueError(
                        f"Cannot move from {old.status} to {self.status}"
                    )
        super().save(*args, **kwargs)
```

13. Top 30 paper-exam syntax fragments to overlearn

1. `class Foo(models.Model):`
2. `name = models.CharField(max_length=100)`
3. `student = models.ForeignKey(User, on_delete=models.CASCADE, related_name='logs')`
4. `def __str__(self): return self.name`
5. `class Meta: model = Foo; fields = '__all__'`
6. `def validate(self, data): ... return data`
7. `raise serializers.ValidationError("msg")`
8. `serializer = FooSerializer(data=request.data)`
9. `if serializer.is_valid(): serializer.save()`
10. `return Response(serializer.data, status=status.HTTP_201_CREATED)`
11. `Foo.objects.filter(...).exclude(...)`
12. `permission_classes = [IsAuthenticated]`
13. `@receiver(post_save, sender=Foo)`
14. `def handler(sender, instance, created, **kwargs):`
15. `if not created and instance.status == 'Approved':`
16. `send_mail(subject, body, from_email, [to_email])`
17. `self.client.login(username='x', password='y')`
18. `self.assertEqual(response.status_code, 201)`
19. `User.objects.create_user(username='x', password='y')`
20. `path('api/foo/', FooView.as_view())`
21. `const [foo, setFoo] = useState(initialValue)`
22. `useEffect(() => { ... }, [])`
23. `fetch('/api/x/').then(r => r.json()).then(setFoo)`
24. `fetch(url, {method:'POST', headers:{'Content-Type':'application/json'}, body: JSON.stringify(data)})`
25. `e.preventDefault()`
26. `setFormData({...formData, [e.target.name]: e.target.value})`

27. `{items.map(i => <li key={i.id}>{i.name}</li>)}`
28. `{flag && <Component />}`
29. `<input value={formData.x} onChange={handleChange} name="x" />`
30. `import os; SECRET_KEY = os.environ.get('SECRET_KEY')`

If you can write these 30 lines from memory, you will pass the practical.

---

14. Concept paragraphs you may be asked (4 marks each)

Each "concept" question in AITS was 4 marks. Aim for ~4 short sentences, one mark each. Drafts you can adapt:

REST APIs

A REST API is an HTTP interface exposing resources (e.g. `/api/logs/`) and operating on them using standard verbs (GET, POST, PUT, PATCH, DELETE). It returns structured data — almost always JSON — so the React frontend can fetch and render data without sharing code or a database with the backend. This separation lets frontend and backend evolve independently. In ILES, the React UI fetches the student's weekly logs from a Django REST endpoint instead of querying the database directly.

Input validation

Input validation rejects bad data before it touches the database, protecting against bugs (wrong types, missing required fields) and security issues (SQL injection, oversized payloads). DRF serializers centralise this — `validate_<field>` for single fields and `validate` for cross-field rules — and return clear 400-level errors. The frontend then surfaces those errors to the user so they can correct them, instead of failing silently. In ILES this means a weekly log cannot be saved without a week number, activities, and a positive hours value.

useState + onChange in React forms

`useState` gives a component a piece of memory: a current value plus a setter function. Binding the input's `value` to that state and its `onChange` to the setter makes it a controlled input — React owns the value and re-renders the input on every keystroke. This is essential because it lets you validate as the user types, disable Submit until the form is valid, and reset the form after submission.

Automated backend testing

Automated tests run your code against a known input and check the output, catching regressions before they reach users. They fail even when code "looks correct" because the test exposes a hidden assumption — e.g. an unauthenticated request silently allowed through, or a duplicate evaluation being saved. In ILES, tests should cover the four state transitions, RBAC (students can't approve logs), and the scoring formula.

REST and the Django/React split

The Django backend handles authentication, business rules (state transitions, weighted scoring) and persistence, and exposes its data as JSON over REST. React owns the UI: it fetches that JSON, renders forms and dashboards, and sends user actions back as POST/PATCH/DELETE requests. Neither side imports the other; they communicate only over HTTP. This is what allows the same backend to power a web app, a mobile app, or a CLI later.

useEffect for data fetching

`useEffect(fn, deps)` runs `fn` after render. Passing `[]` as `deps` makes it run exactly once on mount — the standard pattern for "load data when the page opens." Without a dependency array, the effect

re-runs on every render and you get an infinite fetch loop. Updating state inside the effect triggers a re-render, which is fine because the empty deps array stops the effect from running again.

---

15. The 60-minute pre-exam drill

Spend an hour today doing this with a pen and a blank sheet:

1. Write the full `WeeklyLog` model from memory, including imports.
2. Write a `WeeklyLogSerializer` with one single-field validator and one cross-field validator.
3. Write a DRF `APIView` with `get` and `post`. No peeking.
4. Write a `post_save` signal that emails on status change to Approved.
5. Write a test class with `setUp`, login, POST, and `assertEqual(...,201)`.
6. Write a complete React `LogForm` with controlled inputs, submit, success, error, loading.
7. Write a complete React `LogList` with `useState([])`, `useEffect(...,[])`, fetch, map with `key`.
8. Write `settings.py` snippet with `DEBUG`, `ALLOWED_HOSTS`, `DATABASES`, `STATIC_ROOT` for production.
9. Write a Procfile and a `.env` for Heroku.
10. Write a 4-mark concept paragraph on REST APIs and another on input validation.

If any of those 10 things take longer than 6 minutes to write, **redo only that one** until it does.